Lecture 03: combinational logic

Goals

- recognize and describe the behavior of AND, OR, and NOT gates
- recognize the meaning of the little circle to negate circuit inputs and outputs
- define "combinational logic"
- use the *sum-of-products* method to build a circuit to implement an arbitrary Boolean expression
- describe the purpose of a *multiplexer* and implement one of arbitrary size
- describe the purpose of a *decoder* and implement one of arbitrary size

last time we looked at transistors and boolean logic

we saw how transistors worked, where the gate controls whether the source is connected to the drain

then we briefly summarized this algebra that George Boole invented 150+ years ago

combining the two, we showed how transistors can be used to implement boolean logic operations (spec

this required us, however, to abstract away certain aspects of the transistors

we didn't worry about the exact voltage level, we assumed presence of voltage was a 1 and absence of

now it's time to abstract away all the details of transistors

we've shown how OR and NOT can be implemented using transistors (and you'll implement AND on the

so we're going to hide those implementation details behind the idea of *logic gates*

so instead of a bunch of transistors, we're going to use not-gates (often called inverters), and-gates, or-g

(draw on board)

(refer to Gates component collection in Logisim)

note that a circle on an input or output pin indicates negation

can also have multiple inputs

(draw cascading 2-input AND-gates on board, show abstraction as single 3-input AND-gate)

just like we combined transistors, we can combine these gates to perform more complex operations

we divide the kinds of circuits we can build with gates into two groups: circuits that remember and circuits

for a circuit that doesn't remember, its output is always a combination of its inputs at any given instant

hence, these are called *combinational logic* (sometimes called *combinatorial logic*)

(we'll get to the circuits that do remember, called storage elements, soon)

let's look at some examples of combinational logic

for our first foray into combinational logic, let us design a circuit that has two inputs, A and B, and one ou

we want this circuit to allow us to choose which input (A or B) is passed to the output:

if we choose A and A is 0, the output should be 0

if we choose A and A is 1, the output should be 1

if we choose B and B is 0, the output should be 0

if we choose B and B is 1, the output should be 1

how are we going to choose, though?

we need another input

let's call this other input "select" and decree that, when select is 0, we're choosing the value of A and wh
the value of B

that's a human-language description of the behavior we want, but it probably isn't clear how to construct

one way to begin to address that is to translate our human-language description into a truth table

```
 A | B | sel | out
---+---+-----+-----
 0 | 0 |  0  |  0
 0 | 0 |  1  |  0
 0 | 1 |  0  |  0
 0 | 1 |  1  |  1
 1 | 0 |  0  |  1
 1 | 0 |  1  |  0
 1 | 1 |  0  |  1
 1 | 1 |  1  |  1
```

now, identify every row where the output is 1: there are 4 of them

for each of these rows, construct the Boolean expression, containing all input variables, that makes the r

for instance, when A is 0, B is 1, and sel is 1, the output is 1

the corresponding Boolean expression would be "NOT A AND B AND sel"

because when A is 0, B is 1, and sel is 1, the expression "NOT A AND B AND sel" is true

for this truth table, the four expressions are:

```
NOT A AND B AND sel

A AND NOT B AND NOT sel

A AND B AND NOT sel

A AND B AND sel
```

finally, we just wrap each expression in parentheses, and connect them with ORs:

```
(NOT A AND B AND sel)
        OR
(A AND NOT B AND NOT sel)
        OR
(A AND B AND NOT sel)
        OR
(A AND B AND sel)
```

the result is a Boolean expression that is true exactly when the truth table is true, and false otherwise (be
the precise circumstances in which it should be true and included no others)

you might wonder if there is a more concise Boolean expression

in this case, the answer is yes

check out these two rows:

A	B	sel	out
0	1	1	1
1	1	1	1

note that the value of A doesn't matter!

as long as B is 1 and sel is 1, the output is going to be 1, and we can ignore the value of A

so we can instead describe those two rows with the expression "B AND sel"

likewise, consider these two rows:

A	B	sel	out
1	0	0	1
1	1	0	1

here, it's the value of B that doesn't matter: as long as A is 1 and sel is 0, we can safely produce a 1

thus these two rows can be described by the expression "A AND NOT sel"

so now our full expression is "(A AND NOT sel) OR (B AND sel)"

and our task becomes to implement an equivalent circuit

fortunately, the Boolean operators NOT, AND, and OR translate directly to circuit elements

if we take just the sub-expression "B AND sel", this tells us we need an AND gate, where one of the inpu

likewise, "A AND NOT sel" tells us we need another AND gate where one of the inputs is A and the other being passed through an inverter

since these two subexpressions are connected by the OR operator, that tells us we just need to take the and connect them to the inputs of a single OR gate

and the output of the OR gate is the value of "out"

like so: mux_2.circ

this circuit we have just built is called a *multiplexer*

and it is an example of combinational logic because, given a particular set of inputs, we always know exa

you will use them quite frequently in this class when you need to implement choice

and indeed this circuit element is so widely used that Logisim has one built in: it's found in the Plexers lib

we might ask ourselves whether we could produce a multiplexer with more inputs: can we choose among inputs?

we certainly can!

the first question to ask is how many select wires do we need for a certain number of inputs?

a single input wire, which can only carry two different values (0 and 1) is insufficient to select among 4 di

if we add a second wire, how many inputs can we choose from?

the most straight-forward way to answer this question is to list all the possible combinations of values on

```
0 0
0 1
1 0
1 1
```

four different combinations means we can specify four different values, which means that we can choose

more generally, if there are *x* ways of doing one task and *y* ways of doing a second task, there are *x* time
tasks

in this case, this means that, since there are two ways of picking the value of the first select wire and two
the second select wire, there are 2 times 2 = 4 ways of picking the pair of wires

if we had 4 wires, there would be 2^4 = 16 ways

in general, if we have *n* select wires, we can choose from among (up to) 2^n inputs

reversing the logic, if we need to choose from among *m* things, we need at least log_2 m select wires

(if you're fuzzy on "log_2 m", it means "find x where 2^x == m")

so we've decided we need 2 select wires to choose from 4 inputs

the next task is to write down a truth table, translate it to a Boolean expression, reduce it if we can, and t

I encourage you to do this on your own and compare to my solution:

mux_4.circ

to return to the process we used above…

it's a method called "sum of products"

here it is in succinct form:

- write out the desired truth table
- ignore all the rows for which the output is 0
- for each row where the output is 1, write a Boolean expression that describes it (eg, if the row is A=0,
  expression would be "NOT A AND B AND sel")
- combine all the expressions you just wrote using OR—this encapsulates the idea that the circuit shou
  those conditions holds (and no others)
- simplify the expression where possible
- translate the Boolean operations directly to the corresponding gates
- profit

the multiplexer is, as I said, extremely widely used and you need to know what it does and how it works

another very popular piece of combinational logic is a *decoder*

a decoder has *n* inputs and *2^n* outputs

at any time, exactly one of those outputs is 1 and the rest are 0

the particular output set to 1 is identified by the *n* inputs

(remember that *n* inputs can produce 2^n different combinations, which is why the decoder has 2^n outp

these are useful for figuring out what to do based on some input

decoder_2to4.circ

Definitions

The following definitions introduced in this lecture are fair-game for future quizzes. You will be expected t
provided in these lecture notes.

- combinational logic

Last modified: 09/18/2026 23:04:46